

Project Executable Files

Date	04 July 2026
Team ID	
Project Name	Smart Lender
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide (instructions to run the project)	Yes
3	requirements.txt / package.json / dependency file	Yes
4	Database schema / seed files (if applicable)	Yes
5	Environment configuration file (.env.example or similar)	Yes
6	Deployed application URL (if hosted)	No
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes 239 tests passed
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

SmartLender_Project_codes/

```

|
|
| - app.py                # Main Flask application (548 lines)
| - train_model.py        # ML training pipeline — 5 models, CV, charts (708 lines)
| - requirements.txt      # Python dependencies
| - pytest.ini            # Pytest configuration
| - README.md             # Project overview & setup guide
| - .gitignore
|
| - data/
|   | - train.csv          # 614 labelled loan applicant records (source dataset)
|   |
|   | - model/             # Auto-generated after running train_model.py
|   |   | - best_model.pkl  # Trained best model (Logistic Regression — 86.18%)
|   |   | - scaler.pkl      # Fitted StandardScaler
|   |   | - feature_encoders.pkl # LabelEncoders for 6 categorical features
|   |   | - target_encoder.pkl # LabelEncoder for Loan_Status (Y/N)
|   |   | - feature_columns.json # Ordered feature column list (14 features)
|   |   |   | - feature_importances.json # Feature importance scores for Risk Breakdown panel
|   |   |   | - model_comparison.json # Accuracy/CV/AUC metrics for all 5 models
|   |   |
|   | - static/
|   |   | - css/
|   |   |   | - style.css # Full design system with dark mode
|   |   |   |
|   |   |   | - js/
|   |   |   |   | - main.js # Dark mode, EMI calc, animated counters, drag-drop
|   |   |   |   | - reports/ # Auto-generated after running train_model.py
|   |   |   |     | - model_comparison.png
|   |   |   |     | - feature_importances.png
|   |   |   |     | - roc_curves.png
|   |   |   |     | - confusion_matrices.png

```

```

|   ├── calibration_curves.png
|   ├── income_vs_loan.png
|   ├── approval_breakdown.png
|   └── credit_history_impact.png
|
|
└─ templates/
    ├── base.html           # Shared navbar, dark mode toggle, footer
    ├── index.html          # Home — loan application form + EMI calculator
    ├── result.html         # Prediction result — SVG gauge, feature bars, EMI
    ├── batch.html          # Bulk CSV upload & batch scoring
    ├── history.html        # Session-based application history log
    ├── reports.html        # Analytics chart gallery
    └── about.html          # Model methodology & metrics
|
└─ tests/
    ├── __init__.py
    ├── conftest.py         # Shared pytest fixtures
    ├── test_preprocessing.py # Unit tests: build_feature_row, _calc_emi (29 tests)
    ├── test_routes.py      # Integration tests: all HTML routes (37 tests)
    ├── test_api.py         # API tests: /api/predict, /api/emi (51 tests)
    ├── test_model_pipeline.py # ML pipeline unit tests (38 tests)
    ├── test_model_performance.py # Performance regression tests (28 tests)
    ├── test_batch.py       # Batch processing tests (30 tests)
    └── test_session_history.py # Session history tests (15 tests)
  
```

Total: 239 tests — ALL PASSING ■

Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	http://127.0.0.1:5000

Login Credentials (Demo)	No authentication required — application is open access
Platform / Hosting Provider	Local Python/Flask development server
Repository Link	https://github.com/vallepupavani2005-ctrl/smart-lender-laon
Demo Video Link	https://drive.google.com/file/d/1NELbIQ--JwxPq_DS6o9gHrFiliksg2EO/view?usp=sharing

Step 4: Run Instructions

SmartLender - SmartBridge:

The complete source code of the application is available in the GitHub repository under the folder:

""SmartLender_Project_codes""

PRE REQUISITES :

- Python 3.10 or higher installed and available in PATH
- pip package manager available
- Approximately 200 MB free disk space
- Internet connection (for initial pip install only)

STEP 1 — Clone the Repository:

Open **Command Prompt, PowerShell, or Terminal**.

Execute the following command to clone the repository.

```
</> Bash  
git clone https://github.com/YaswanthGanesh1234/SmartLender-SmartBridge.git
```

After cloning is complete, navigate into the repository.

```
</> Bash  
cd SmartLender-SmartBridge
```

STEP 2 — Navigate to the Project Folder:

The repository contains multiple folders.

The complete Smart Lender project is located inside:

```
SmartLender_Project_codes
```

Navigate into this folder.

```
</> Bash
```

```
cd SmartLender_Project_codes
```

STEP 3 — Install Project Dependencies :

Install all required Python libraries.

```
</> Bash
```

```
pip install -r requirements.txt
```

The following major libraries will be installed:

- | Flask
- | pandas
- | NumPy
- | scikit-learn
- | XGBoost
- | matplotlib
- | joblib

Installation may take several minutes depending on the internet connection.

STEP 4 — Generate Machine Learning Model Files :

Before launching the web application, the machine learning pipeline must be executed.

Run:

```
</> Bash
```

```
python train_model.py
```

This step performs the following tasks:

- | Loads and preprocesses the dataset
- | Trains all machine learning models
- | Compares model performance
- | Selects the best performing model
- | Saves trained model artifacts
- | Generates analytics charts
- | Stores all required files inside the **model** folder

This process generally takes **30–60 seconds**.

STEP 5 — Launch the Flask Web Application :

Start the application using:

```
</> Bash  
  
python app.py
```

The terminal should display output similar to:

```
PS C:\Users\Yaswanth Ganesh\Desktop\SmartLender> python app.py  
* Serving Flask app 'app'  
* Debug mode: off  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on all addresses (0.0.0.0)  
* Running on http://127.0.0.1:5000  
* Running on http://10.94.106.188:5000  
Press CTRL+C to quit  
127.0.0.1 - - [01/Jul/2026 08:31:30] "GET / HTTP/1.1" 200 -  
127.0.0.1 - - [01/Jul/2026 08:31:34] "GET /static/css/style.css HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:31:43] "GET /static/js/main.js HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:32:13] "POST /predict HTTP/1.1" 200 -  
127.0.0.1 - - [01/Jul/2026 08:32:13] "GET /static/css/style.css HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:32:13] "GET /static/js/main.js HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:32:22] "GET /reports HTTP/1.1" 200 -  
127.0.0.1 - - [01/Jul/2026 08:32:22] "GET /static/css/style.css HTTP/1.1" 304 -  
127.0.0.1 - - [01/Jul/2026 08:32:22] "GET /static/js/main.js HTTP/1.1" 304 -
```

STEP 6 — Open the Application :

Visit:

```
http://127.0.0.1:5000
```

STEP 6 — (Optional) Running the Test Suite:

```
</> Bash
```

```
python -m pytest tests/ -v
```

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Session history not persistent — resets on server restart or cookie expiry	By design for local demo. Integrate SQLite for production persistence.
2	Batch CSV download tied to session — opening /batch/download in a new tab without prior upload redirects to /batch	Click 'Download CSV' directly on batch results page. Do not navigate away first.
3	XGBoost overfits on 614-row dataset — 100% train accuracy but 81.3% test	Logistic Regression (86.18%) is auto-selected by the model selection logic.